Stochastic Gradient Descent (SGD): Visual Deep Dive

Core Theory, Mathematical Formulations, and 2D Loss Surface Trajectories

1. Definition & Mathematical Foundations

Stochastic Gradient Descent (SGD) is a foundational iterative optimization algorithm in machine learning. Unlike classical Batch Gradient Descent (BGD), which evaluates gradients across the entire training dataset of size N before updating model weights, SGD calculates the parameter update using only a single randomly sampled training instance (x_i, y_i) at each step.

Mathematical Update Formulations:

· Parameter Update Rule: w_{t+1} = w_t - · · · ·L_i(w_t)

· Unbiased Expectation: E_{i}[·L_i(w)] = ·J(w) (Stochastic sample gradient equals full gradient)

· Batch Regimes: Classical SGD (B = 1) | Mini-Batch GD (1 < B < N) | Full-Batch GD (B = N)

2. Loss Surface Trajectory Analysis

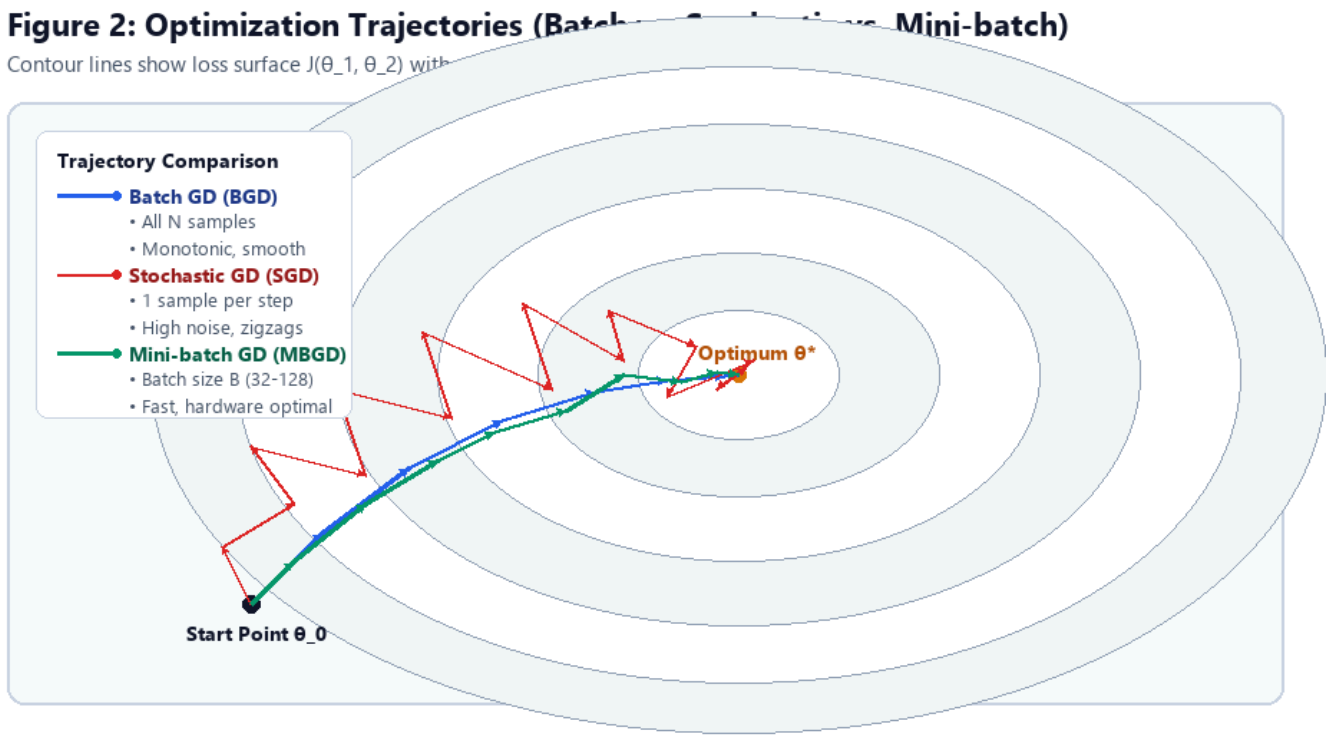


Figure 1 Caption: Comparison of optimization paths on elliptical 2D loss contours.
Blue = Batch GD (monotone & smooth) | Red = SGD (noisy stochastic exploration) | Green = Mini-batch GD (hardware optimal).

3. Theoretical Trade-offs: Noise as Regularization

Why does SGD succeed in deep neural networks despite its high gradient noise? In non-convex loss landscapes containing multiple local minima and saddle points, the deterministic trajectory of Batch GD frequently gets trapped in shallow, suboptimal local minima. In contrast, the stochastic fluctuations of single-sample SGD provide an inherent annealing-like noise that enables parameter vectors to escape poor basins and discover wider, flatter minima with superior generalization capability.

Variant	Batch Size	Computational Cost / Step	Trajectory Characteristics
Batch GD	N (Full Dataset)	O(N · d) · High compute	Monotonic, direct descent; trapped in saddle points
Stochastic GD	1 Sample	O(d) · Fast minimal compute	Erratic zigzag trajectory; escapes shallow minima
Mini-batch GD	B (32 to 256)	O(B · d) · Tensor vectorized	Balanced path; standard industry choice

Hyperparameter Tuning & Convergence Behavior

Learning Rate Schedules, Momentum Acceleration, and Verification Targets

4. Learning Rate Schedules & Momentum Mechanics

To guarantee convergence toward the global minimum, the learning rate ? must decay across training iterations. Standard schedules include Step Decay and Exponential Annealing. Furthermore, adding a Momentum term dampens perpendicular oscillations while accelerating descent along long ravines.

Analytical Formulations for Convergence:

· Step Decay Schedule: ·_t = ·_0 · (·)^{ floor(epoch / k) } [In this module: ·_0 = 0.1, · = 0.5, k = 10 epochs]

· Momentum Velocity Update: v_{t+1} = · · v_t + · · ·L_i(w_t) [Momentum coefficient · = 0.90]

· Parameter Step with Momentum: w_{t+1} = w_t - v_{t+1}

5. Empirical Loss Curves Across Learning Rates

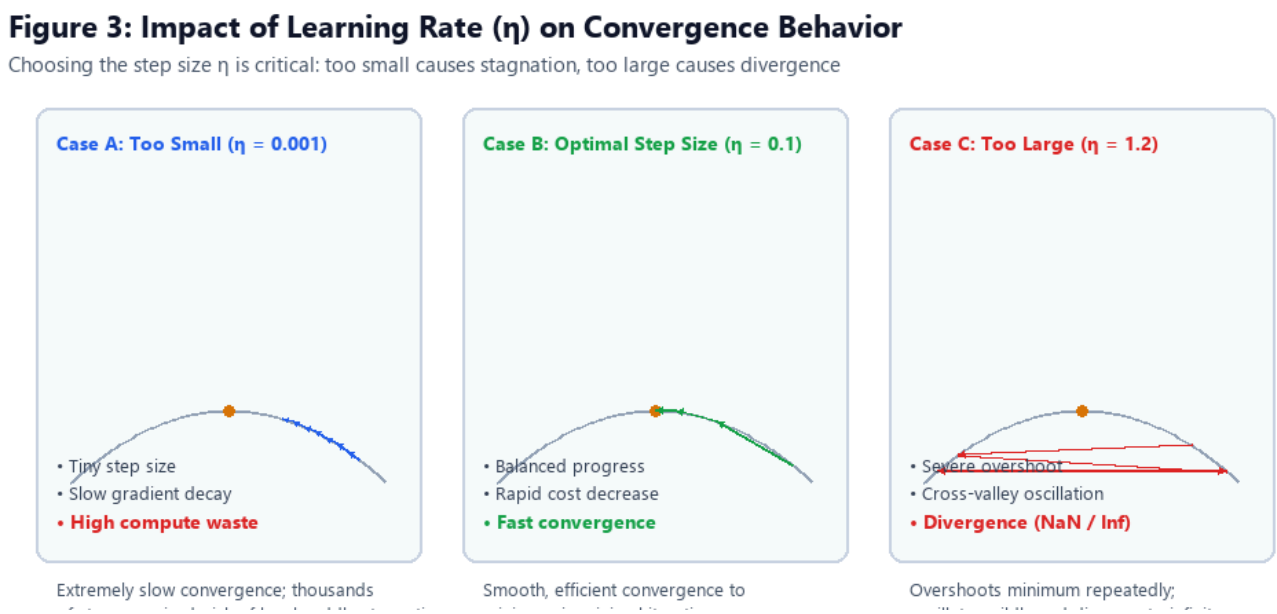


Figure 2 Caption: Empirical loss trajectories under three learning rate regimes.
Panel A (Left): Small · = 0.001 (stagnation) | Panel B (Center): Optimal · = 0.1 (convergence) | Panel C (Right): Large · = 1.2 (divergence).

6. Concrete Numerical Walkthrough (Exact Step Calculation)

Worked Example for Model Verification:

· Setup: Single-weight linear model y_hat = w · x, with Squared Loss L = 0.5 · (y_hat - y)^2

· Inputs: Initial weight w_0 = 1.00, input sample x = 2.0, target ground truth y = 4.0, learning rate · = 0.10

· Loss & Gradient: Prediction y_hat = 1.00 · 2.0 = 2.00 · Error (y_hat - y) = -2.00 · ·_w L = (-2.00) · 2.0 = -4.00

· Updated Weight: w_1 = w_0 - · · ·_w L = 1.00 - (0.10 · -4.00) = 1.00 + 0.40 = 1.40

7. Grounded Verification Targets for Wizora-AI Testing

Key Facts for Automated Q&A and MCQ Validation:

1. Classical SGD Batch Size: Exactly 1 training example per parameter update.
2. Trajectory Profile: SGD produces an erratic zigzag trajectory; Batch GD yields a smooth monotonic descent.
3. Decay Factor in Schedule: The step decay schedule multiplies the learning rate by exactly 0.5 every 10 epochs.
4. Momentum Advantage: Dampens high-frequency oscillations perpendicular to the ravine and accelerates along the base.
5. Numerical Checkpoint: An initial weight of 1.00 with x=2, y=4, and ·=0.10 updates to exactly 1.40.